

Lesson 7 Structured Note: C Implementation — Node, Global Pointer, Display, and Main

0. Current Position in the Course

You are now in **Lesson 7** of the stack curriculum.

This lesson belongs to:

Phase 2: Linked-List Stack

Before Lesson 7, you learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .

Lesson 7 continues the linked-list stack.

This lesson is mainly about writing a **complete C program** for a linked-list stack using:

- `struct Node`
- a global `top` pointer
- `push()`
- `pop()`
- `display()`
- a `main()` function for testing

In simple words:

Lessons 5 and 6 taught the linked-list stack pieces. Lesson 7 puts the pieces together into one working C program.

1. Lesson 7 Goal

By the end of this lesson, you should be able to:

- Define a linked-list stack node in C.
 - Declare and initialize a global `top` pointer.
 - Write `push(int x)` to insert a value at the top.
 - Write `pop()` to remove and return the top value.
 - Write `display()` to print the stack from top to bottom.
 - Test the stack using `main()`.
 - Dry-run the code step by step.
 - Explain why `push()` and `pop()` are $O(1)$.
 - Explain why `display()` is $O(n)$.
 - Understand why `malloc()` and `free()` are needed.
 - Avoid common pointer and memory mistakes.
-

2. Quick Review: What Is a Stack?

A stack is a linear data structure.

It follows this rule:

LIFO = Last In, First Out

This means:

The last value inserted into the stack is the first value removed from the stack.

Example:

```
push(10);  
push(20);  
push(30);
```

Logical stack:

```
Top -> 30
      20
      10
```

The last inserted value is `30`.

So if we call:

```
pop();
```

The value removed will be:

```
30
```

Important rule:

In a stack, insertion and deletion happen only at the top.

3. Array Stack vs Linked-List Stack

You already learned array stacks earlier.

In an array stack, `top` is an integer index.

Example:

```
Index: 0   1   2
Value: 10  20  30
        top = 2
```

Here:

```
S[top] == S[2] == 30
```

So the top element is `30`.

In a linked-list stack, `top` is different.

In a linked-list stack:

`top` is a pointer to the first node.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

Comparison:

Concept	Array stack	Linked-list stack
Storage	Array	Nodes
Meaning of <code>top</code>	Index	Pointer
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Full condition	<code>top == size - 1</code>	<code>malloc()</code> returns <code>NULL</code>
Push location	End/right side of array	Front of linked list
Pop location	End/right side of array	Front of linked list
Fixed size needed?	Yes	No fixed size at start

Important idea:

The implementation changes, but the stack rule stays the same.

Both array stacks and linked-list stacks follow **LIFO**.

4. What Lesson 7 Is Really About

Lesson 7 is not mainly about a new stack rule.

You already know the stack rule:

```
Last In, First Out
```

You also already know the main linked-list stack operations:

```
push()  
pop()
```

Lesson 7 focuses on assembling a complete C linked-list stack program.

The main code parts are:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int x);
int pop();
void display();

int main() {
    // test stack here
}
```

So Lesson 7's central question is:

How do we write the full C program for a linked-list stack using a global `top` pointer?

5. Required Header Files

A C program for a linked-list stack needs these header files:

```
#include <stdio.h>
#include <stdlib.h>
```

5.1 Why `stdio.h`?

`stdio.h` is used for input and output functions.

In this lesson, we use:

```
printf()
```

Example:

```
printf("Stack Overflow\n");
```

5.2 Why `stdlib.h`?

`stdlib.h` is used for dynamic memory functions.

In this lesson, we use:

```
malloc()  
free()
```

`malloc()` creates new nodes in heap memory.

`free()` releases nodes after they are removed.

Important:

A linked-list stack creates nodes dynamically, so `stdlib.h` is required.

6. The Node Structure

A linked-list stack is made of nodes.

Each node stores two things:

1. The actual value.
2. A pointer to the next node.

In C, the node is written like this:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

6.1 Meaning of Each Field

Field	Simple meaning
<code>data</code>	Stores the actual stack value.
<code>next</code>	Stores the address of the next node.

Example node:

```
[ data = 10 | next = NULL ]
```

This means:

- The node stores the value `10`.
- The node does not point to another node.
- So its `next` field is `NULL`.

6.2 Why `next` Uses `struct Node *`

This line:

```
struct Node *next;
```

means:

`next` is a pointer to another `struct Node`.

That is how nodes are connected.

Example:

```
[30 | next] -> [20 | next] -> [10 | NULL]
```

The `next` pointer creates the chain.

7. The Global `top` Pointer

After defining the node, we declare:

```
struct Node *top = NULL;
```

This creates a pointer named `top`.

In a linked-list stack:

`top` points to the first node of the linked list.

The first node is the top of the stack.

7.1 Empty Stack

At the beginning, the stack is empty.

So we write:

```
struct Node *top = NULL;
```

This means:

```
top -> NULL
```

There is no node in the stack.

7.2 Non-Empty Stack

After pushing values, the stack may look like this:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here:

- `top` points to the node containing `30`.
- `30` is the top element.
- `20` is below `30`.
- `10` is at the bottom.
- The last node points to `NULL`.

7.3 Why `top` Is Global in This Lesson

In Lesson 7, `top` is declared outside all functions:

```
struct Node *top = NULL;
```

Because it is outside all functions, it is global.

That means functions like these can access it directly:

```
push()  
pop()  
display()
```

This makes the beginner C program simpler.

Important note:

A global `top` pointer is convenient, but it is less protected. Later, in a C++ class version, `top` can be hidden inside a class.

8. Empty Condition in a Linked-List Stack

A linked-list stack is empty when:

```
top == NULL
```

Why?

Because `top` is a pointer.

If `top` points to nothing, there is no first node.

Example:

```
top -> NULL
```

This means the stack has no elements.

Do not use this for a linked-list stack:

```
top == -1
```

That is for array stacks, not linked-list stacks.

Correct linked-list empty condition:

```
top == NULL
```

9. Full Condition in a Linked-List Stack

An array stack becomes full when the array is full.

Array stack full condition:

```
top == size - 1
```

But a linked-list stack does not have a fixed array size.

A linked-list stack grows by creating new nodes in heap memory.

So a linked-list stack is full only when heap memory is not available.

That is checked by `malloc()`.

If this line fails:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

then `malloc()` returns:

```
NULL
```

So linked-list stack overflow happens when:

```
t == NULL
```

Simple meaning:

Linked-list stack overflow means memory allocation failed.

10. The `push(int x)` Function

10.1 Meaning of `push(x)`

`push(x)` inserts a new value at the top of the stack.

Example before pushing:

```
top -> 20 -> 10 -> NULL
```

Call:

```
push(30);
```

After pushing:

```
top -> 30 -> 20 -> 10 -> NULL
```

The new value `30` becomes the top.

11. `push(int x)` Code

```
void push(int x) {  
    struct Node *t;  
  
    t = (struct Node *)malloc(sizeof(struct Node));  
  
    if (t == NULL) {  
        printf("Stack Overflow\n");  
    } else {  
        t->data = x;  
        t->next = top;  
        top = t;  
    }  
}
```

12. Explanation of `push(int x)`

Step 1: Create a temporary pointer

```
struct Node *t;
```

`t` will point to the new node.

At this point, no node has been created yet.

Step 2: Allocate memory for one node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for enough memory to store one `struct Node`.

Simple meaning:

Create one new node dynamically.

Step 3: Check if memory allocation failed

```
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

If `t == NULL`, the heap could not provide memory.

That means the stack cannot grow.

This is called:

Stack Overflow

Step 4: Store the value

```
t->data = x;
```

This stores the value `x` inside the new node.

Example:

```
push(30);
```

Then:

```
t->data = 30;
```

Step 5: Link the new node to the old top

```
t->next = top;
```

This is one of the most important lines.

It makes the new node point to the previous top node.

Example before:

```
top -> 20 -> 10 -> NULL
```

New node:

```
t -> 30
```

After:

```
t->next = top;
```

The situation becomes:

```
t -> 30 -> 20 -> 10 -> NULL
top -> 20 -> 10 -> NULL
```

The new node is now connected to the old stack.

Step 6: Move `top` to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

Final result:

```
top -> 30 -> 20 -> 10 -> NULL
```

13. The Most Important `push()` Pattern

The key push pattern is:

```
t->next = top;  
top = t;
```

Meaning:

1. The new node points to the old top.
2. `top` moves to the new node.

This order matters.

Correct order

```
t->next = top;  
top = t;
```

Wrong order

```
top = t;  
t->next = top;
```

Why is the wrong order bad?

Because if you move `top` first, you may lose the old stack connection.

It may also make the new node point to itself.

Important rule:

First connect the new node to the old stack. Then move `top`.

14. `push()` Dry Run

Start:

```
top -> NULL
```

Operation 1: push(10)

Create new node:

```
t -> 10
```

Set:

```
t->next = top;
```

Since `top` is `NULL`:

```
t -> 10 -> NULL
```

Move `top`:

```
top = t;
```

Result:

```
top -> 10 -> NULL
```

Operation 2: push(20)

Current stack:

```
top -> 10 -> NULL
```

Create new node:

```
t -> 20
```

Set:

```
t->next = top;
```

Now:

```
t -> 20 -> 10 -> NULL
```

Move `top`:

```
top -> 20 -> 10 -> NULL
```

Operation 3: `push(30)`

Current stack:

```
top -> 20 -> 10 -> NULL
```

Create new node:

```
t -> 30
```

Set:

```
t->next = top;
```

Now:

```
t -> 30 -> 20 -> 10 -> NULL
```

Move `top`:

```
top -> 30 -> 20 -> 10 -> NULL
```

Final stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

Logical view:

```
Top -> 30  
      20  
      10
```

15. The `pop()` Function

15.1 Meaning of `pop()`

`pop()` removes and returns the top value.

Example before pop:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
pop();
```

The top value is:

```
30
```

After pop:

```
top -> 20 -> 10 -> NULL
```

Returned value:

```
30
```

16. pop() Code

```
int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }

    return x;
}
```

17. Explanation of pop()

Step 1: Create a temporary pointer

```
struct Node *p;
```

`p` will point to the node that will be deleted.

Step 2: Set a default return value

```
int x = -1;
```

`x` stores the value that will be returned.

If the stack is empty, there is no valid value to return.

So `-1` is used as a simple error value.

Important note:

Using `-1` as an error value is simple, but it has a limitation. If `-1` is a valid stack value, the caller cannot easily tell whether `-1` means an error or a real popped value.

For beginner code, this method is acceptable.

Step 3: Check underflow

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

If `top == NULL`, the stack is empty.

There is no node to remove.

This error is called:

Stack Underflow

Step 4: Save the old top node

```
p = top;
```

Now `p` points to the node that will be removed.

Example:

```
p  
|  
v  
top -> 30 -> 20 -> 10 -> NULL
```

Step 5: Save the data

```
x = p->data;
```

This saves the top value before deleting the node.

If the top node contains `30`, then:

```
x = 30;
```

Step 6: Move `top` to the next node

```
top = top->next;
```

Now the second node becomes the new top.

Before:

```
top -> 30 -> 20 -> 10 -> NULL
```

After:

```
top -> 20 -> 10 -> NULL
```

The old node containing `30` is no longer part of the stack.

Step 7: Free the old top node

```
free(p);
```

This releases the memory used by the removed node.

This is necessary because the node was created using `malloc()`.

Important rule:

Every node removed by `pop()` should be freed using `free()`.

Otherwise, the program wastes heap memory.

That problem is called a memory leak.

Step 8: Return the saved value

```
return x;
```

If the removed value was `30`, then `pop()` returns:

```
30
```

18. The Most Important `pop()` Pattern

The key pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Meaning:

1. Save the old top node.
2. Save the value inside it.
3. Move `top` to the next node.
4. Free the old top node.
5. Return the saved value.

This order matters.

Correct idea

```
p = top;  
top = top->next;  
free(p);
```

Wrong idea

```
free(top);  
top = top->next;
```

Why is the wrong idea unsafe?

Because after freeing `top`, using `top->next` is invalid.

Important rule:

Move `top` safely before freeing the old node.

19. `pop()` Dry Run

Start:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
pop();
```

Step 1: Check if stack is empty

```
top == NULL
```

This is false because `top` points to node `30`.

Step 2: Save old top

```
p = top;
```

Now:

```
p -> 30
```

Step 3: Save data

```
x = p->data;
```

So:

```
x = 30
```

Step 4: Move top

```
top = top->next;
```

Now:

```
top -> 20 -> 10 -> NULL
```

Step 5: Free old node

```
free(p);
```

The node containing `30` is deleted from heap memory.

Step 6: Return value

```
return x;
```

So `pop()` returns:

```
30
```

Final stack:

```
top -> 20 -> 10 -> NULL
```

20. The `display()` Function

20.1 Meaning of `display()`

`display()` prints all stack elements from top to bottom.

Example stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

The display output should be:

```
30 20 10
```

Why?

Because the stack should be displayed starting from the top.

21. `display()` Code

```
void display() {
    struct Node *p = top;

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }

    printf("\n");
}
```

22. Explanation of `display()`

Step 1: Create a temporary pointer

```
struct Node *p = top;
```

`p` starts at the top node.

We use `p` so that we do not move the real `top` pointer.

Important:

Never move `top` just to display the stack.

If you move `top`, you will damage the stack.

Step 2: Traverse until `NULL`

```
while (p != NULL)
```

The loop continues as long as `p` points to a valid node.

When `p` becomes `NULL`, the traversal is finished.

Step 3: Print the current node's data

```
printf("%d ", p->data);
```

This prints the value inside the current node.

Step 4: Move to the next node

```
p = p->next;
```

This moves `p` forward by one node.

Example:

```
30 -> 20 -> 10 -> NULL
```

The pointer moves like this:

```
30, then 20, then 10, then NULL
```

23. `display()` Does Not Change the Stack

`display()` only reads the stack.

It does not remove nodes.

It does not insert nodes.

It does not change `top`.

Before display:

```
top -> 30 -> 20 -> 10 -> NULL
```

After display:

```
top -> 30 -> 20 -> 10 -> NULL
```

The stack remains the same.

24. Complete Lesson 7 C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int x) {
    struct Node *t;

    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}

int pop() {
    struct Node *p;
```

```

int x = -1;

if (top == NULL) {
    printf("Stack Underflow\n");
} else {
    p = top;
    x = p->data;
    top = top->next;
    free(p);
}

return x;
}

void display() {
    struct Node *p = top;

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }

    printf("\n");
}

int main() {
    push(10);
    push(20);
    push(30);

    printf("Stack before pop: ");
    display();

    printf("Popped value: %d\n", pop());

    printf("Stack after pop: ");
    display();

    return 0;
}

```

25. Explanation of the Complete Program

Step 1: Include header files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` gives us `printf()`.

`stdlib.h` gives us `malloc()` and `free()`.

Step 2: Define the node

```
struct Node {
    int data;
    struct Node *next;
};
```

Each node stores:

- one integer value
 - one pointer to the next node
-

Step 3: Create global `top`

```
struct Node *top = NULL;
```

At the beginning:

```
top -> NULL
```

The stack is empty.

Step 4: Push values

```
push(10);  
push(20);  
push(30);
```

After these operations:

```
top -> 30 -> 20 -> 10 -> NULL
```

Step 5: Display before pop

```
printf("Stack before pop: ");  
display();
```

The output is:

```
Stack before pop: 30 20 10
```

Step 6: Pop one value

```
printf("Popped value: %d\n", pop());
```

The top value is 30.

So pop() returns:

```
30
```

After pop:

```
top -> 20 -> 10 -> NULL
```

Step 7: Display after pop

```
printf("Stack after pop: ");  
display();
```

The output is:

```
Stack after pop: 20 10
```

26. Expected Output

The program prints:

```
Stack before pop: 30 20 10  
Popped value: 30  
Stack after pop: 20 10
```

27. Full Dry Run

Program operations:

```
push(10);  
push(20);  
push(30);  
display();  
pop();  
display();
```

Initial state

```
top -> NULL
```

Operation 1: push(10)

Create node 10 .

Since the old `top` is `NULL`, node `10` points to `NULL`.

Move `top` to node `10`.

Result:

```
top -> 10 -> NULL
```

Operation 2: `push(20)`

Create node `20`.

Node `20` points to old top, which is node `10`.

Move `top` to node `20`.

Result:

```
top -> 20 -> 10 -> NULL
```

Operation 3: `push(30)`

Create node `30`.

Node `30` points to old top, which is node `20`.

Move `top` to node `30`.

Result:

```
top -> 30 -> 20 -> 10 -> NULL
```

Operation 4: `display()`

Start pointer `p` at `top`.

p -> 30

Print 30.

Move p to next node.

p -> 20

Print 20.

Move p to next node.

p -> 10

Print 10.

Move p to next node.

p -> NULL

Stop.

Output:

30 20 10

The stack does not change.

Operation 5: pop()

Current stack:

top -> 30 -> 20 -> 10 -> NULL

The top node contains 30.

Save old top:

```
p -> 30
```

Save data:

```
x = 30
```

Move top:

```
top -> 20 -> 10 -> NULL
```

Free old node:

```
node 30 is deleted
```

Return:

```
30
```

Operation 6: `display()` again

Current stack:

```
top -> 20 -> 10 -> NULL
```

Output:

```
20 10
```

28. Dry Run Summary Table

Step	Operation	Return/output	Stack after operation
0	Start	—	<code>top -> NULL</code>
1	<code>push(10)</code>	—	<code>top -> 10 -> NULL</code>

Step	Operation	Return/output	Stack after operation
2	<code>push(20)</code>	—	<code>top -> 20 -> 10 -> NULL</code>
3	<code>push(30)</code>	—	<code>top -> 30 -> 20 -> 10 -> NULL</code>
4	<code>display()</code>	<code>30 20 10</code>	<code>top -> 30 -> 20 -> 10 -> NULL</code>
5	<code>pop()</code>	returns <code>30</code>	<code>top -> 20 -> 10 -> NULL</code>
6	<code>display()</code>	<code>20 10</code>	<code>top -> 20 -> 10 -> NULL</code>

29. Memory Behavior

A linked-list stack uses heap memory.

Every successful `push()` creates one new node using:

```
malloc()
```

Every successful `pop()` deletes one node using:

```
free()
```

During `push()`

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This creates memory for one new node.

During `pop()`

```
free(p);
```

This releases memory for the removed node.

Important:

If you use `malloc()` to create nodes, you should use `free()` to release removed nodes.

30. Overflow and Underflow

30.1 Overflow

Overflow means trying to push when the stack cannot accept another value.

In a linked-list stack, this happens when heap memory is unavailable.

Condition:

```
t == NULL
```

Code:

```
t = (struct Node *)malloc(sizeof(struct Node));  
  
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

Simple meaning:

Stack overflow means `malloc()` failed.

30.2 Underflow

Underflow means trying to pop when the stack is empty.

In a linked-list stack, the stack is empty when:

```
top == NULL
```

Code:

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

Simple meaning:

Stack underflow means there is no node to remove.

31. Array Overflow vs Linked-List Overflow

Stack type	Overflow condition	Simple meaning
Array stack	<code>top == size - 1</code>	The array has no empty slot.
Linked-list stack	<code>malloc()</code> returns <code>NULL</code>	Heap memory cannot create a new node.

32. Array Underflow vs Linked-List Underflow

Stack type	Underflow condition	Simple meaning
Array stack	<code>top == -1</code>	No active array element exists.
Linked-list stack	<code>top == NULL</code>	No node exists.

33. Time Complexity

Operation	Time complexity	Reason
<code>push()</code>	<code>O(1)</code>	Adds one node at the front.
<code>pop()</code>	<code>O(1)</code>	Removes one node from the front.
<code>display()</code>	<code>O(n)</code>	Visits every node.

34. Why `push()` Is `O(1)`

`push()` does a fixed number of steps:

1. Allocate one node.
2. Store the value.
3. Link the new node to the old top.
4. Move `top`.

It does not scan the full list.

So:

push() = O(1)

35. Why `pop()` Is $O(1)$

`pop()` directly removes the node pointed to by `top`.

It does not search for the top node.

It already has direct access to the top node through:

```
top
```

So:

```
pop() =  $O(1)$ 
```

36. Why `display()` Is $O(n)$

`display()` must print every node.

Example:

```
top -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL
```

To print the full stack, it must visit:

```
50, 40, 30, 20, 10
```

If there are `n` nodes, `display()` visits `n` nodes.

So:

```
display() =  $O(n)$ 
```

37. Space Complexity

Stack storage

If the stack has `n` values, it has `n` nodes.

So total stack storage is:

`O(n)`

`push()` auxiliary space

One `push()` operation uses only a few variables:

```
struct Node *t;
```

So auxiliary space is:

`O(1)`

`pop()` auxiliary space

One `pop()` operation uses:

```
struct Node *p;  
int x;
```

So auxiliary space is:

`O(1)`

`display()` auxiliary space

`display()` uses one pointer:

```
struct Node *p;
```

So auxiliary space is:

`0(1)`

Even though `display()` takes `0(n)` time, it uses only `0(1)` extra space.

38. Complexity Summary Table

Function	Time	Auxiliary space	Reason
<code>push()</code>	<code>0(1)</code>	<code>0(1)</code>	Adds one node at the top.
<code>pop()</code>	<code>0(1)</code>	<code>0(1)</code>	Removes one node from the top.
<code>display()</code>	<code>0(n)</code>	<code>0(1)</code>	Traverses all nodes.

39. Operation Effects Summary

Operation	What it does	Changes stack?
<code>push(x)</code>	Inserts <code>x</code> at the top.	Yes
<code>pop()</code>	Removes and returns the top value.	Yes
<code>display()</code>	Prints all nodes from top to bottom.	No

Important:

- `push()` changes `top`.
- `pop()` changes `top` and frees a node.
- `display()` uses a temporary pointer and does not change `top`.

40. Common Beginner Mistakes

Mistake 1: Thinking `top` is an index

Wrong idea:

`top` stores an index

Correct idea:

In a linked-list stack, top is a pointer.

In an array stack:

```
top == -1
```

In a linked-list stack:

```
top == NULL
```

Mistake 2: Using `top == -1` in a linked-list stack

Wrong:

```
if (top == -1)
```

Correct:

```
if (top == NULL)
```

Reason:

`top` is a pointer, not an integer index.

Mistake 3: Reversing the push links

Wrong:

```
top = t;  
t->next = top;
```

Correct:

```
t->next = top;  
top = t;
```

Reason:

The new node must first point to the old top. Then `top` can move to the new node.

Mistake 4: Forgetting to check `malloc()`

Wrong:

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

Problem:

If `malloc()` fails, `t` is `NULL`.

Then this is unsafe:

```
t->data = x;
```

Correct:

```
t = (struct Node *)malloc(sizeof(struct Node));

if (t == NULL) {
    printf("Stack Overflow\n");
} else {
    t->data = x;
    t->next = top;
    top = t;
}
```

Mistake 5: Forgetting underflow check before `pop()`

Wrong:

```
p = top;
x = p->data;
```

Problem:

If `top == NULL`, then `p` is also `NULL`.

Accessing this is invalid:

```
p->data
```

Correct:

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

Mistake 6: Freeing the node too early

Wrong:

```
free(top);  
top = top->next;
```

Problem:

After freeing `top`, using `top->next` is unsafe.

Correct:

```
p = top;  
top = top->next;  
free(p);
```

Mistake 7: Forgetting to free popped nodes

Wrong idea:

```
Just move top and ignore the old node.
```

Problem:

The old node remains allocated in heap memory.

That causes a memory leak.

Correct:

```
free(p);
```

Mistake 8: Moving `top` inside `display()`

Wrong idea:

```
while (top != NULL) {  
    printf("%d ", top->data);  
    top = top->next;  
}
```

Problem:

This destroys the stack because it moves the real `top` pointer.

Correct:

```
struct Node *p = top;  
  
while (p != NULL) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

Use a temporary pointer for traversal.

41. Key Rules to Remember

Rule 1: Linked-list `top` is a pointer

```
struct Node *top = NULL;
```

Rule 2: Empty linked-list stack

```
top == NULL
```

Rule 3: Push pattern

```
t->data = x;  
t->next = top;  
top = t;
```

Rule 4: Pop pattern

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Rule 5: Display pattern

```
p = top;  
while (p != NULL) {  
    print p->data;  
    p = p->next;  
}
```

Rule 6: `malloc()` creates memory

```
malloc(sizeof(struct Node))
```

Rule 7: `free()` releases memory

```
free(p);
```

42. Beginner Checkpoints

You understand Lesson 7 if you can answer these:

1. What does `struct Node` store?
2. Why does a node need a `next` pointer?
3. What does `top = NULL` mean?
4. Why is `top` a pointer in a linked-list stack?
5. What does `push(30)` do to this stack?

```
top -> 20 -> 10 -> NULL
```

1. Why must `t->next = top` happen before `top = t`?
2. What does `pop()` return from this stack?

```
top -> 30 -> 20 -> 10 -> NULL
```

1. Why must `pop()` use `free(p)`?
2. Why does `display()` use a temporary pointer `p`?
3. Why is `push()` `O(1)`?
4. Why is `display()` `O(n)`?

43. Checkpoint Answers

1. What does `struct Node` store?

It stores:

- the data value
- the pointer to the next node

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

2. Why does a node need a `next` pointer?

Because the `next` pointer connects one node to the next node.

Without `next`, the nodes would not form a linked list.

3. What does `top = NULL` mean?

It means the stack is empty.

```
top -> NULL
```

4. Why is `top` a pointer in a linked-list stack?

Because it stores the address of the top node.

In a linked-list stack, nodes are connected by addresses.

5. What does `push(30)` do here?

Before:

```
top -> 20 -> 10 -> NULL
```

After:

```
top -> 30 -> 20 -> 10 -> NULL
```

It creates a new node containing `30` and makes it the new top.

6. Why must `t->next = top` happen before `top = t`?

Because the new node must first connect to the old stack.

Then `top` can safely move to the new node.

7. What does `pop()` return here?

Given:

```
top -> 30 -> 20 -> 10 -> NULL
```

`pop()` returns:

30

After pop:

top -> 20 -> 10 -> NULL

8. Why must `pop()` use `free(p)`?

Because the removed node was created using `malloc()`.

If we do not free it, memory is wasted.

9. Why does `display()` use a temporary pointer `p`?

Because `display()` needs to move through the list.

If it moved `top`, the stack would be destroyed.

So it uses:

```
struct Node *p = top;
```

10. Why is `push()` $O(1)$?

Because it only creates one node and updates a few pointers.

It does not scan the list.

11. Why is `display()` $O(n)$?

Because it must visit every node to print every value.

If there are `n` nodes, it visits `n` nodes.

44. Practice Tasks

Practice 1: Draw the stack

Draw the stack after these operations:

```
push(5);  
push(12);  
push(7);
```

Expected final structure:

```
top -> 7 -> 12 -> 5 -> NULL
```

Practice 2: Dry-run one pop

Start:

```
top -> 7 -> 12 -> 5 -> NULL
```

Call:

```
pop();
```

Answer:

```
Returned value: 7  
Final stack: top -> 12 -> 5 -> NULL
```

Practice 3: Predict display output

Given:

```
top -> 40 -> 30 -> 20 -> 10 -> NULL
```

What does `display()` print?

Answer:

40 30 20 10

Practice 4: Identify the bug

What is wrong with this code?

```
void display() {
    while (top != NULL) {
        printf("%d ", top->data);
        top = top->next;
    }
}
```

Answer:

It moves the real `top` pointer.

That destroys the stack.

Correct version:

```
void display() {
    struct Node *p = top;

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

Practice 5: Add one more pop

Given this program:

```
push(10);
push(20);
push(30);
```

```
printf("%d\n", pop());  
printf("%d\n", pop());  
display();
```

Trace:

After pushes:

```
top -> 30 -> 20 -> 10 -> NULL
```

First `pop()` returns:

```
30
```

Stack:

```
top -> 20 -> 10 -> NULL
```

Second `pop()` returns:

```
20
```

Stack:

```
top -> 10 -> NULL
```

`display()` prints:

```
10
```

45. Lesson 7 Final Summary

Lesson 7 teaches how to build a complete linked-list stack program in C.

The main structure is:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

The stack starts empty:

```
struct Node *top = NULL;
```

The empty condition is:

```
top == NULL
```

`push()` inserts a new node at the front:

```
t->next = top;  
top = t;
```

`pop()` removes the front node:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);
```

`display()` prints from top to bottom:

```
p = top;  
while (p != NULL) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

After:

```
push(10);  
push(20);  
push(30);
```

The stack becomes:

```
top -> 30 -> 20 -> 10 -> NULL
```

`display()` prints:

```
30 20 10
```

`pop()` returns:

```
30
```

Then the stack becomes:

```
top -> 20 -> 10 -> NULL
```

The most important idea:

A linked-list stack uses the front node as the top because insertion and deletion at the front are $O(1)$.

If you can explain how `top` changes after every `push()` and `pop()`, you understand Lesson 7.